# Assignment 2: A Real Life Competition

Group 162: Alessandro Bertoncini 2903205, Antonio Pascarella 2895210, and Vesper Kukler 2747734

Vrije Universiteit Amsterdam, Amsterdam, The Netherlands

## 1 Business Understanding

### 1.1 Business Context

We are focusing on a recommender system, a specific AI-based tool that suggests products, items, or content to users. In this case, it is a hotel-filtering tool that presents different accommodation options based on the user's requests.

Today, most travelers rely on platforms like Expedia or Booking to select the best hotel in terms of quality and price. For this reason, it is crucial for hospitality businesses to appear on these websites. In addition, ranking is a key factor: users are unlikely to scroll through all the listings, so the options shown at the top are far more likely to be chosen.

### 1.2 Business Problem

The core challenge is to identify the factors that influence users' choices. In particular, this understanding enables booking platforms to present options that closely match users' preferences and requirements. As a result, Expedia (or Booking, etc.) can achieve higher conversion rates, hotels can increase their revenue, and customers are more satisfied. In other words, it is a win-win scenario: the user quickly finds exactly what they are looking for with the minimum number of clicks.

### 1.3 Data Mining Objectives

At this stage, we will focus on the following objectives:

- *Understand, Analyse and Elaborate Data*: We must first clarify the overall context: what data do we have? Does it accurately describe the situation? Does it require further enrichment, and if so, in what way?
- *Feature Selection*: Determine the most relevant attributes that can be used as features for training and evaluating the model.
- *Build the Model (and iterate if necessary)*: Because we need to generate a detailed ranking list in response to users' queries, we will develop a classification model capable of assigning one of the following labels to hotel properties: **booked**, **clicked**, **ignored**. The results will then be benchmarked against Normalized Discounted Cumulative Gain (NDCG)@5 calculated per query and averaged over all queries with the values weighted by the $\log_2$ function.

### 1.4 Success Criteria and Constraints

"`The winner will be rewarded with the fame and glory of winning the 2026VU Data Mining Techniques cup.`" This sounds quite enticing, doesn't it? To this end, we strive to obtain the highest possible score according to the NDCG evaluation metric. At the same time, we also aim to secure strong precision and recall values.

### 1.5 Related Work

One of the most compelling solutions proposed for this specific problem uses a collection of machine learning models whose predictions are then aggregated through several ensemble methods [1]. The following key ideas from that work could be integrated into our approach:

- **Validation Set and Class Balance:** around 10% of the training data is set aside as a validation set to compare and tune the models. In addition, because tree-based methods and neural networks are sensitive to class imbalance, the training data is constructed to have a 1:1 ratio of positive to negative examples. Here, a positive example indicates that the hotel was booked or clicked, while a negative example indicates that it was ignored.
- **Composite Features:** this is a standard step in feature engineering. Certain features become more informative when combined. In this case, a new feature called *count_ window* is defined to jointly represent both "number of rooms" and "booking lead time" in a single value, potentially enabling some models to better learn the interaction between these two aspects.
- **Per-Country Splitting:** to accelerate the training of tree-based models, the dataset is partitioned by country, yielding 172 separate subsets. An independent model is then trained on each subset. This not only reduces training time but also aligns with the intuition that hotels within the same country are likely to exhibit similar behaviour.
- **Handling Missing Values:** within each country-specific subset, the first quartile of a feature's distribution is computed and used to impute missing values. Compared to mean imputation, this approach is more robust in the presence of outliers.

## 2 Data Understanding

### 2.1 Dataset Overview

The training dataset contains a total of 4,958,347 rows, corresponding to 199,795 distinct searches conducted over approximately 241 days (from 1 November 2012 to 30 June 2013). Reporting the number of rows with missing values is not very informative, as every record has at least one missing field. It is also worth noting that the training set includes some attributes that are not present in the test set:

Furthermore, there are 129,113 unique hotels, of which 51,768 belong to a major chain while the remaining 77,345 are independent ones. They are spread

| | | Training set only |
|---|---|---|
| position | Integer | Hotel position on Expedia's search results page. This is only provided for the training data, but not the test data |
| click_bool | Boolean | 1 if the user clicked on the property, 0 if not |
| booking_bool | Boolean | 1 if the user booked the property, 0 if not |
| gross_booking_usd | Float | Total value of the transaction. This can differ from the price_usd due to taxes, fees, conventions on multiple day bookings and purchase of a room type other than the one shown in the search |

across 172 countries, whereas customers engaged with Expedia from 210 different countries.

### 2.2 Target Distribution and Query Structure

Each query returns between 5 and 38 results, with a median of 29. The two target labels are highly imbalanced: only 4.47% of rows are clicked and 2.79% are booked. Per query, most users click on a single property (186,764 out of 199,795 queries with at least one click) and book at most one (138,390 queries contain a booking, the remaining 61,405 do not). This imbalance motivates the 1:1 down-sampling strategy proposed by [1] and is consistent with the use of NDCG@5 as evaluation metric, since most of the relevance mass is concentrated on the top results.

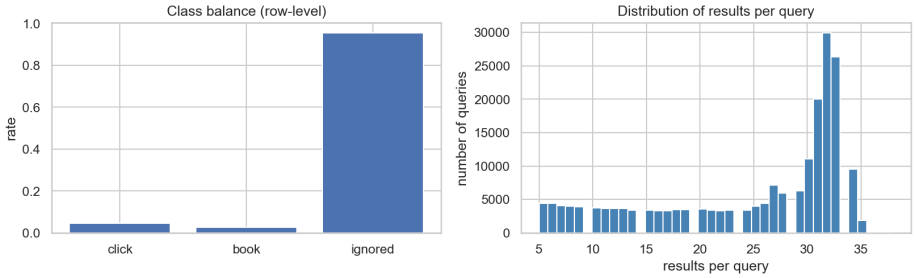


**Fig. 1.** Class balance at row level (left) and distribution of the number of results per query (right).

### 2.3 Position Bias

About 29.6% of queries are returned in random order (`random_bool`=1), while the remaining ones use Expedia's default sort. Comparing click-through and

booking rates across positions for the two groups isolates the position bias from the true relevance signal (Fig. 2). For Expedia-sorted queries the engagement decays sharply with position; for random queries the decay is much milder. We will exploit this difference when training the ranker, either by relying on the random-sorted subset for evaluation or by applying a position-aware weighting.

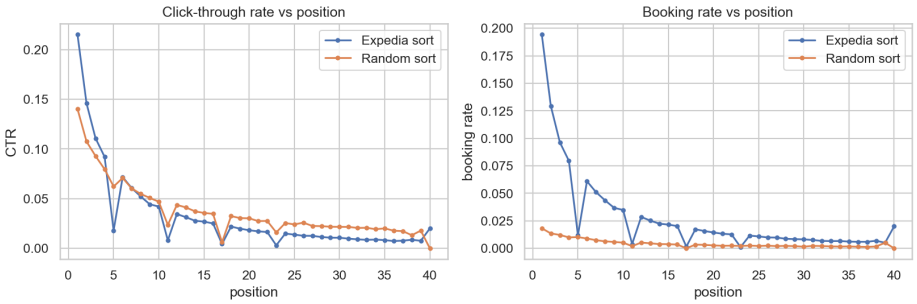


**Fig. 2.** Click-through rate (left) and booking rate (right) by position, split by sort type.

### 2.4 Price Patterns

The `price_usd` column has a long right tail: the median is $122, the 99th percentile is $599, but the maximum reaches $1.97 M and 2,871 rows exceed $5,000. There are also 31 rows with a price of zero. These extremes are likely encoding errors and will be clipped or removed during preparation. More importantly, the *relative* price within a query is a much stronger signal than the absolute one: ranking the cheapest option first within each search yields a clear monotonic decay of CTR and booking rate with the price rank (Fig. 3). We therefore plan to engineer a `price_rank_in_query` feature.

### 2.5 Property Quality

Engagement increases with both star rating and review score, peaking at four-star properties (CTR 5.4%, booking rate 3.3%), and dips slightly for five-star ones, probably because of the higher price. The `prop_brand_bool` flag has only a small effect (chain hotels are booked slightly more often: 2.92% vs 2.57%), while the `promotion_flag` produces a large lift (CTR 6.0% vs 4.0%, booking rate 3.9% vs 2.5%). These features are clearly useful for ranking.

### 2.6 Missing Values

Missingness varies a lot across columns (Fig. 5). The competitor columns (`comp*_rate`, `comp*_inv`, `comp*_rate_percent_diff`) are missing in 52% to

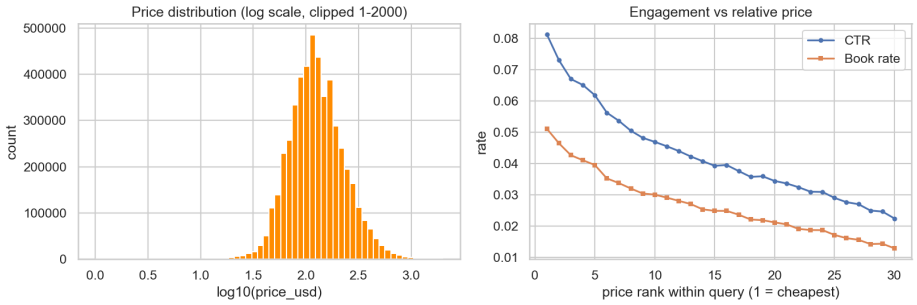


**Fig. 3.** Price distribution on a log scale (left) and engagement by price rank within the query (right).

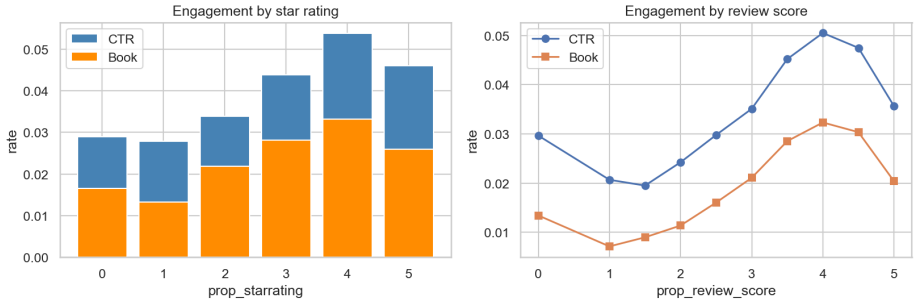


**Fig. 4.** Click-through and booking rate by star rating (left) and review score (right).

98% of rows, since not every competitor has data for every search. Visitor history (`visitor_hist_starrating`, `visitor_hist_adr_usd`) is missing 95% of the time, and `srch_query_affinity_score` is absent in 94% of rows. The distance `orig_destination_distance` is missing in 32% of rows and `prop_location_score2` in 22%. We will treat missingness as informative (adding indicator flags) and impute the remaining values using the per-country first quartile, as suggested by [1].

## 3 Data Preparation

The data preparation step turns the raw training and test sets into two clean tables that share exactly the same set of features. All transformations are applied through a single helper function so that the pipeline can be reproduced on the test set without leakage from the validation data. The final dataset has 82 columns, of which 30 are engineered.

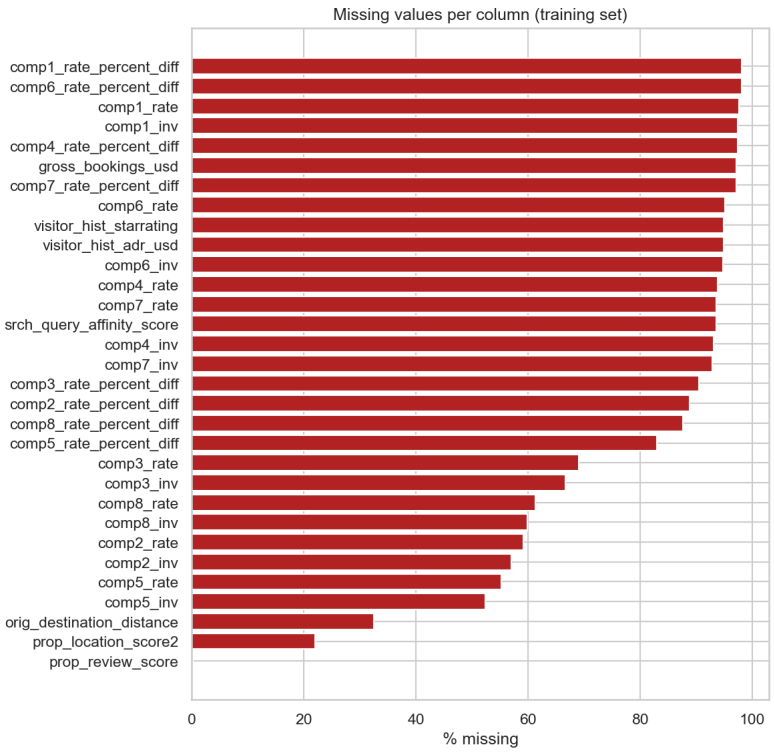


**Fig. 5.** Percentage of missing values per column in the training set.

### 3.1 Outlier Handling

The exploratory analysis showed that `price_usd` contains values up to about \$1.97 M and 31 zero-priced rows. Both extremes are clearly encoding errors and would dominate any regression that involves the price. We therefore clip the column to the realistic interval $[1, 5000]$ and store the cleaned values in a new column `price_clipped`. We also keep a logarithmic version, `log_price` $= \log(1 +$ `price_clipped`$)$, since the price distribution is heavily skewed and many models work better on a logarithmic scale. We deliberately do not drop the outlying rows because each query needs to keep all of its results to compute relative features such as ranks within the query.

### 3.2 Missing Value Handling

The exploratory analysis revealed that missingness is widespread but not random: it is concentrated in competitor columns, in visitor history and in `srch_query_affinity_score`. We adopt three different strategies depending on the nature of the column.

- **Indicator flags.** For every column with a non-trivial fraction of missing values we add a binary flag `<col>_missing`. This way the model can learn that the very fact of being missing carries information, for example a user without history is rarely a returning customer.
- **Per-country first-quartile imputation.** For numerical attributes such as `prop_review_score`, `prop_location_score2`, `orig_destination_distance`, `srch_query_affinity_score` and the visitor history columns, we replace the missing values with the first quartile computed inside the same `prop_country_id`. This idea is taken from the reference paper [1] and is more robust than mean imputation because the distributions are skewed and contain outliers. If a country has no observation for a given column we fall back to the global first quartile.
- **Zero imputation for competitor differences.** The columns `comp1_rate_percent_diff` to `comp8_rate_percent_diff` are missing whenever the competitor itself is not present, so we fill them with zero, which corresponds to no price difference.

The first-quartile lookup is computed on the training set and then applied to both training and test, so the test set never influences the imputation values.

### 3.3 Feature Engineering

The new features are organised in four groups, all motivated by findings from the exploratory analysis or by the reference paper.

- **Per-query relative features.** The ranking task is intrinsically relative: what matters is how a property compares to its peers in the same search. We compute the price rank inside the query (`price_rank_in_query`) and z-scores within the query for the price and for the three quality scores `prop_starrating`, `prop_review_score` and `prop_location_score2`. The exploratory analysis already showed that the relative price is much more informative than the absolute one.
- **Composite features.** We add normalisations of the price by the duration of the stay (`price_per_night`) and by the number of adults (`price_per_person`), the `count_window` feature suggested by [1] as the product of room count and booking lead time, the gap between the current price and the user's historical average daily rate (`price_diff_history`), the gap between the property star rating and the user's historical preference (`star_diff_history`), and the difference between the current log price and the historical log price of the property (`hist_price_delta`).
- **Competitor aggregates.** The eight competitor columns are very sparse, so instead of using them individually we summarise them with three counts: `n_comp_expedia_cheaper`, `n_comp_expedia_pricier` and `n_comp_available`. The exploratory analysis confirmed that the first one has a clear monotonic effect on the booking rate.
- **Simple flags.** A `domestic` flag is set when the visitor and the property share the country, and `has_history` marks the rows where the user has previous activity on Expedia. Both proved to correlate with the booking rate.

### 3.4 Relevance Label and Validation Split

The competition is a learning-to-rank problem evaluated with NDCG@5. Following the standard convention for this dataset we build a graded relevance label that combines the two original targets:

$$\texttt{relevance} = \begin{cases} 5 & \text{if the property was booked,} \\ 1 & \text{if the property was clicked but not booked,} \\ 0 & \text{otherwise.} \end{cases}$$

The values 5 and 1 are the same used by the official Kaggle evaluation script and reflect the fact that a booking is much more valuable than a click. Building this label upfront also lets us apply standard learning-to-rank objectives such as LambdaRank later on.

To reproduce the evaluation procedure offline we hold out 10% of the queries as a validation set, using `GroupShuffleSplit` on `srch_id`. Splitting at query level (and not at row level) is essential: it prevents the same search from being partially seen during training and ensures that NDCG@5 on the validation set is computed exactly as it will be on the test set. The resulting split keeps the click rate (4.5%) and the booking rate (2.8%) very close to the full training set, which means the validation sample is representative.

### 3.5 Persistence and Test Set Alignment

Finally, the same pipeline is applied to the test set using the imputation lookup learned on the training data, and the three resulting tables (training, validation and test) are saved as Parquet files inside a `processed/` folder. Saving in Parquet rather than CSV is a deliberate choice: it keeps column types intact (so the new `int8` flags do not get cast back to `float64`) and reduces the disk footprint by about a factor of three, which is convenient since the training table alone has 4.96 M rows and 82 columns.

## 4 Modeling and Evaluation

We are a group of three, so we decided to split the modeling work in three: each of us implemented a different model and then we combined the predictions through an ensemble, an idea taken from the reference paper [1]. Working like this gives us three independent points of view on the same problem and an easy way to measure how much they actually agree.

### 4.1 Setup and Evaluation Metric

All three models are trained on the same prepared training set (`processed/train.parquet`, around 4.46 million rows over 179,815 queries) and evaluated on the held-out validation set (495,086 rows over 19,980 queries). The feature list

is built once and reused by every model: we keep the 79 numeric columns of the prepared dataset, dropping the targets (`click_bool`, `booking_bool`, `relevance`, `gross_bookings_usd`), the row identifiers (`srch_id`, `prop_id`, `date_time`) and the leakage column `position`. The metric is NDCG@5, computed per query and averaged: for each search we sort the candidate hotels by predicted score, take the top five, and compute

$$\text{NDCG@5} = \frac{1}{Z} \sum_{i=1}^{5} \frac{\text{rel}_i}{\log_2(i+1)},$$

where $Z$ is the ideal DCG@5 of the same query. We implement the formula in a small helper `ndcg_at_5` that we call after each model to get a directly comparable number.

### 4.2 Model 1: Logistic Regression

The first model is a class-balanced logistic regression on the booking target (`booking_bool`). It is the simplest pointwise baseline: it scores each (query, hotel) pair independently, ignoring the fact that the task is a ranking. We standardise the features with `StandardScaler`, set `class_weight='balanced'` to compensate for the 2.8% positive rate, and use the predicted booking probability as the score. This model is fast (a few minutes), easy to read and gives us a sanity check: any more sophisticated model should beat it on NDCG@5. On the validation set it reaches NDCG@5 = 0.3466.

### 4.3 Model 2: LightGBM with LambdaRank

The second model is a gradient-boosted ensemble of decision trees trained with the LambdaRank objective. Unlike logistic regression, LambdaRank is a listwise method: at each step it looks at the whole list of candidates of a search and rewards the model when it puts a more relevant hotel above a less relevant one. We use the graded `relevance` label (5 for booked, 1 for clicked, 0 otherwise) and pass the per-query group sizes through the `group=` argument of `LGBMRanker`. We train up to 500 trees with `learning_rate=0.05` and `num_leaves=63`, and rely on early stopping on the validation NDCG@5 (patience 30) so we do not have to tune the number of trees by hand. The reference paper uses LambdaMART with similar hyperparameters and reports it as the best single model, so we expect this to be our strongest base model. On the validation set it reaches NDCG@5 = 0.3935 after 398 trees.

### 4.4 Model 3: Matrix Factorization (SVD)

The third model is a matrix factorization, the technique presented in the Recommender Systems lecture. We build a sparse implicit-feedback matrix $R$ where rows are unique `srch_destination_id` values and columns are unique `prop_id`

values; each cell counts $2 \cdot \#\text{bookings} + 1 \cdot \#\text{clicks}$ observed in the training data. We then run a truncated SVD with $k = 32$ components, which gives us a destination matrix $U \in R^{n_d \times k}$ and a property matrix $V \in R^{n_p \times k}$. The score for a candidate hotel is the dot product $U_d \cdot V_p$ of its destination and property latent vectors. The intuition is that destinations that are typically booked together share latent factors with their popular hotels, so the dot product captures a notion of destination-hotel affinity that the per-row models cannot see. Rows of the test set with an unseen `srch_destination_id` or `prop_id` get a score of zero. On the validation set the SVD alone reaches NDCG@5 = 0.2719, which is the weakest of the three but, as we will see, it still adds useful diversity to the ensemble.

### 4.5 Ensemble

Each model produces a score on a different scale: a probability in $[0, 1]$ for the logistic regression, an unbounded log-odds for LightGBM, and a small dot product for SVD. To make them comparable we apply a per-query z-score normalisation, the same trick used by the reference paper for their winning ensemble. For every search we replace each model's score with $\tilde{s}_m = (s_m - \mu_q)/\sigma_q$, where the mean and standard deviation are computed inside that single search. Then we combine the three normalised scores with a non-negative weighted sum

$$s_{\text{ensemble}} = w_1 \tilde{s}_{\text{LR}} + w_2 \tilde{s}_{\text{LGBM}} + w_3 \tilde{s}_{\text{SVD}},$$

with $w_1 + w_2 + w_3 = 1$. The weights are tuned by a small grid search (step 0.1) over the validation NDCG@5. The best combination is $w_{\text{LGBM}} = 0.8$, $w_{\text{SVD}} = 0.2$, $w_{\text{LR}} = 0.0$, which gives NDCG@5 = 0.3999. The grid search dropped the logistic regression entirely, but kept a non-trivial weight for the SVD: that is exactly the kind of diversity we hoped to see, the SVD complements the ranker by adding destination-level information.

### 4.6 Results

Table 1 summarises the validation NDCG@5 of each model and of the ensemble. The ensemble is the best, beating the strongest single model (LightGBM) by about 0.6 NDCG points. Final test predictions are produced by the ensemble and written to `submission.csv` (4,959,183 rows, 199,549 unique searches), ready to be uploaded to the Kaggle leaderboard.

## 5 Deployment

To address the ethical AI part of the assignment, we investigated whether the final ranking model under-exposes **independent hotels** compared with **chain hotels**. We used `prop_brand_bool` as the group indicator, where `prop_brand_bool`=0 denotes an independent hotel and `prop_brand_bool`=1 denotes a chain hotel. The analysis was performed on the validation set using the final ensemble ranking.

**Table 1.** Validation NDCG@5 for each model and for the ensemble.

| **Model** | **Validation NDCG@5** |
|---|---|
| Logistic Regression | 0.3466 |
| LightGBM with LambdaRank | 0.3935 |
| Matrix Factorization (SVD, $k = 32$) | 0.2719 |
| Ensemble ($w_{\text{LGBM}} = 0.8$, $w_{\text{SVD}} = 0.2$) | **0.3999** |

### 5.1 Bias Detection

Because this is a ranking problem, we focused on **exposure fairness**. In particular, we compared how often independent hotels appear in the candidate set with how often they appear in the top-5 recommendations shown to the user.

Our main fairness metric was the **representation gap** for independent hotels:

$$\text{RepresentationGap}_{\text{ind}} = \text{Top5Share}_{\text{ind}} - \text{CandidateShare}_{\text{ind}}.$$

A negative value means that independent hotels are underrepresented in the top-5 ranking relative to their availability in the candidate set.

We also computed two supporting fairness indicators. First, we measured the **discounted top-5 exposure share**, where higher ranks receive more weight through the same logarithmic discount used in NDCG. Second, we compared the **average rank** of independent and chain hotels. As utility metric, we monitored validation `NDCG@5` to verify that fairness improvements did not substantially reduce ranking quality.

Before mitigation, independent hotels represented `36.74%` of all validation candidates but only `35.13%` of the top-5 ranked results, yielding a representation gap of `-0.0162`. Their discounted top-5 exposure share was `35.19%`, and their average rank was `14.69`, compared with `14.47` for chain hotels. This indicates that the original ranking system systematically placed independent hotels lower in the recommendation list than would be expected from their presence in the candidate set.

### 5.2 Bias Mitigation

We applied a **post-processing** mitigation method based on **query-level adaptive reranking**. We first measured under-exposure inside each query and then applied a larger correction only where independent hotels were underrepresented. For each query $q$, we computed the query-level under-exposure of independent hotels as

$$g_q = \max\Big(0,\ \text{CandidateShare}_{\text{ind},q} - \text{Top5Share}^{\text{base}}_{\text{ind},q}\Big).$$

We then reranked the ensemble scores using

$$s_{\text{fair}} = s_{\text{ensemble}} + \alpha \cdot g_q \cdot \mathbf{1}(\texttt{prop\_brand\_bool} = 0),$$

where $s_{\text{ensemble}}$ is the original ensemble score, $g_q$ is the query-specific under-exposure term, and $\alpha$ is a global hyperparameter. Therefore, queries with no under-exposure receive no fairness correction, while queries with larger under-exposure receive a stronger boost for independent hotels.

We selected $\alpha$ on the validation set using a grid search over values from 0.00 to 1.00 with step size 0.05. Among the values for which the validation `NDCG@5` dropped by at most 0.005, we chose the one that minimised the absolute representation gap for independent hotels. The selected value was `0.50`.

This mitigation is closer to recent work on post-hoc provider fairness, where score adjustments are applied after the original recommender has been trained in order to improve exposure fairness while preserving ranking quality [3]. It is also consistent with recommendation methods that aim to guarantee or increase minimum provider exposure under an explicit utility constraint [2].

This mitigation is attractive in practice because it is simple, transparent, and does not require retraining the original model. It can be added directly after the ranking model has produced its scores.

### 5.3 Effectiveness

The mitigation improved the visibility of independent hotels while preserving most of the original ranking quality. After reranking, the top-5 share of independent hotels increased from `35.13%` to `36.70%`. As a result, the representation gap moved from `-0.0162` to `-0.0004`, almost closing the exposure gap in the top-5.

The discounted top-5 exposure share of independent hotels also improved from `35.19%` to `36.61%`. Moreover, the average rank of independent hotels improved from `14.69` to `14.49`.

In terms of predictive performance, the original ensemble achieved a validation `NDCG@5` of `0.3999`, while the fairness-aware reranking achieved `0.4000`. This shows that a small post-processing correction can reduce under-exposure of independent hotels essentially for free, with no loss in ranking quality on the validation set.

**Table 2.** Validation results before and after fairness-aware reranking.

| Metric | Before | After |
|---|---|---|
| Candidate share (indep.) | 0.3674 | 0.3674 |
| Top-5 share (indep.) | 0.3513 | 0.3670 |
| Representation gap (indep.) | -0.0162 | -0.0004 |
| Discounted top-5 share (indep.) | 0.3519 | 0.3661 |
| Average rank (indep.) | 14.69 | 14.49 |
| Average rank (chain) | 14.47 | 14.59 |
| NDCG@5 | 0.3999 | 0.4000 |

## 6 What We Learned

This assignment gave us deeper insight into how a real ranking task is approached, including exploratory data analysis, feature engineering, model building, evaluation, and fairness-aware post-processing. We especially learned how to translate a business problem into a ranking task, and why query-level structure matters in recommender systems. It further showed that good performance depends not only on the model, but equally on data preparation. A large part of the work did not concern model training itself, but identifying outliers, handling missing values, engineering meaningful features, and making sure that the same pipeline could be applied consistently to both validation and test data.

The main difficulty of the assignment was that the dataset is both large and highly imbalanced. Because only a small fraction of hotel impressions result in clicks or bookings, many seemingly reasonable modeling choices can still lead to weak rankings. In addition, the presence of position bias makes the problem more subtle: user behaviour is influenced not only by hotel relevance, but also by where a hotel is shown in the ranking.

Some outcomes were expected: the ranking-based LightGBM model with LambdaRank performed clearly better than the logistic regression baseline, and query-relative features proved useful for capturing how users compare hotels within the same search. A more unexpected result was that the SVD model was weak on its own but still improved the final ensemble once combined with LightGBM, showing that model diversity can add value. Overall, this assignment taught us that strong results depend not only on the model itself, but also on careful preparation, realistic evaluation, and critical interpretation of the results.

## References

1. Liu, X., Xu, B., Zhang, Y., Yan, Q., Pang, L., Li, Q., Sun, H., Wang, B.: Combination of diverse ranking models for personalized expedia hotel searches. CoRR **abs/1311.7679** (2013), `http://arxiv.org/abs/1311.7679`
2. Lopes, R., Alves, R., Ledent, A., Santos, R.L.: Recommendations with minimum exposure guarantees: A post-processing framework. Expert Systems with Applications **236**, 121164 (2023). https://doi.org/10.1016/j.eswa.2023.121164
3. Tam, J., et al.: Post-hoc provider fairness adaptation via hierarchical exposure alignment. arXiv preprint arXiv:2605.01524 (2026), `https://arxiv.org/html/2605.01524v1`